

□ウィンドウ関数とは

ウィンドウ関数(別名:**OLAP**関数 オーラップかんすう)

OLAPとはOnLine Analytical Processingの略でデータベースを使ってリアルタイム(=オンラインで)データ分析を行う処理のことです。例えば、市場分析、財務諸表作成、マネジメント計画作成など、ビジネスの現場でなくてはならない仕事が含まれます。

ウィンドウ関数は、このOLAP用途のために標準SQLに追加された機能です。すこしざっくり言うと、Excelでデータごとのランクをつけたり、セル範囲を指定してSUM関数やAVERAGE関数を使って計算させる、といったことに近い処理をデータベース上で行うことができる機能です。

MySQLでウィンドウ関数がサポートされるようになったのはMySQL8.0からであり、授業で使用していた教科書では紹介されていませんでした。

□ウィンドウ関数の構文

サンプルを動作させながら学習していきましょう
ウィンドウ関数の構文は少し複雑です

```
<ウィンドウ関数> OVER ([PARTITION BY <列リスト>]  
                        ORDER BY <ソート用列リスト>)
```

※[]部分は省略可能なことを表しています

<ウィンドウ関数>の所には様々な関数が入ります。
重要なキーワードは**PARTITION BY**と**ORDER BY**です。

PARTITION BYとORDER BYを説明する前に
ウィンドウ関数として使える代表的な関数を書いておきます。
ウィンドウ関数には次の2種類に別れます。

- ①集約関数(**SUM, AVG, COUNT, MAX, MIN**)をウィンドウ関数として使う
- ②**RANK, DENSE_RANK, ROW_NUMBER**などのウィンドウ専用関数

②は標準SQLで定義されているOLAP専用の関数です。
①はすでに学習した集約関数ですが、これらは実はウィンドウ関数としても利用できます。
使用する構文によって集約関数になったり、ウィンドウ関数になったりするという感じです。

□RANK関数

手始めにウィンドウ専用関数のRANKを使って、ウィンドウ関数の動作をつかみましょう。
RANKはその名の通り、レコードのランキング(順位)を算出する関数です。

例えば、今まで使ってきたBooksテーブルに含まれる書籍に対して、
出版社(publish)ごとに、価格(price)の高い順に並べたランキング表を作ってみましょう
結果のイメージは以下の通りです。

title	price	publish	ランキング
SQL入門	5500	山田出版	1
PHPドリル	5100	山田出版	2
らくだの観察日記	1100	山田出版	3
フェレットの観察	1000	山田出版	4
ハムスターの観察	900	山田出版	5
かえるの観察日記	800	山田出版	6
あひるの観察日記	700	山田出版	7
PHP5サンプル集	3000	秀和システム	1
JSPリファレンス	1800	秀和システム	2
XMLリファレンス	3200	日経BP	1
SQLリファレンス	2500	日経BP	2
SQLプチブック	1600	日経BP	3
XML辞典	3300	翔泳社	1
PEAR入門	3000	翔泳社	2

山田出版の中で一番高い本『SQL入門』5500円が1位
次に高い『PHP』ドリルが2位、そこから3位4位とつづいて、
秀和システムの2つの本の中でも1位、2位が決まっているのがわかると思います。
出版社ごとに書籍価格の高い順のランキング表になっています。

この結果を求めるSQLはこのように書きます。

```
1  SELECT title,  
2      price,  
3      publish,  
4      RANK() OVER (PARTITION BY publish  
5                      ORDER BY price DESC) AS "ランキング"  
6  FROM books;
```

PARTITION BYは順位をつける対象の範囲を設定しています。
今回、出版社ごとに順位を算出させたいので、publish列を指定しています。

ORDER BYは、どの列をどんな順序で順位付けするかを指定しています。

今回は商品価格の高いものから、順位をつけていきたいので、price DESCと書いてあります。
price ASCと変更すれば、安い本から1位になっていくようになります。

PARTITION BYとORDER BYの作用をイメージしやすいように図に追記したものがこちらです。

ORDER BYの順序 (価格が高い順)				
title	price	publish	ランキング	
SQL入門	5500	山田出版	1	
PHPドリル	5100	山田出版	2	
らくだの観察日記	1100	山田出版	3	
フェレットの観察	1000	山田出版	4	
ハムスターの観察	900	山田出版	5	
かえるの観察日記	800	山田出版	6	
あひるの観察日記	700	山田出版	7	
PHP5サンプル集	3000	秀和システム	1	
JSPリファレンス	1800	秀和システム	2	
XMLリファレンス	3200	日経BP	1	
SQLリファレンス	2500	日経BP	2	
SQLプチブック	1600	日経BP	3	
XML辞典	3300	翔泳社	1	
PEAR入門	3000	翔泳社	2	

PARTITION BYで
区切られたレコードの
集合を『ウィンドウ』
という

PARTITION BYのカット
(商品分類ごと)

出版社ごと

この場合のウィンドウは「窓」ではなく「範囲」を表します。
ウィンドウ関数の名前の由来はここにあります。

テーブルを切り分けるような動作はGROUP BYと似ています。
GROUP BYと違うのは結果が集約されてしまわない所です。
GROUP BYでpublishごとにグループ化したときには、結果が出版社の数である
4レコードに集約されてしまいましたが、PARTITION BYではすべてのレコードが
最終結果に表示されています。

□PARTITION BYは省略可能

上の例で、PARTITION BYとORDER BYが重要な働きをしていると感じたかもしれませんが、実はPARTITION BYは省略可能です。(書かなくてもエラーは出ず、実行可能)

ためにPARTITION BYをなくした下のSQLを実行してみると

```
1  SELECT title,
2      price,
3      publish,
4      RANK() OVER (ORDER BY price DESC) AS "ランキング"
5  FROM books;
```

このような結果が出ます

title	price	publish	ランキング
SQL入門	5500	山田出版	1
PHPドリル	5100	山田出版	2
XML辞典	3300	翔泳社	3
XMLリファレンス	3200	日経BP	4
PHP5サンプル集	3000	秀和システム	5
PEAR入門	3000	翔泳社	5
SQLリファレンス	2500	日経BP	7
JSPリファレンス	1800	秀和システム	8
SQLプチブック	1600	日経BP	9
らくだの観察日記	1100	山田出版	10
フェレットの観察	1000	山田出版	11
ハムスターの観察	900	山田出版	12
かえるの観察日記	800	山田出版	13
あひるの観察日記	700	山田出版	14

最初の例は出版社ごとのランキングでしたが、この結果はテーブル全体での価格のランキングになっています。

イメージとしては、GROUP BYなしでAVG関数などの集約関数を使用したときと同じです。テーブル全体を1つのウィンドウとみなして、全レコードを対象としたランク付けが行われました。

PARTITION BYはあくまで、テーブルを複数の部分(ウィンドウ)に小分けした上でウィンドウ関数を使用したいときのオプションになります。

□ウィンドウ専用関数の種類

直前のSQLの結果をよく見ると

秀和システムの『PHP5サンプル集』と翔泳社の『PEAR入門』がどちらも3000円で同率5位となっています。

そのため6位は存在せず、飛んで7位からランク付けが再開されています。

順位付けの際にはこのような方法はおなじみのやり方になるかもしれませんが、順位が飛ばないようにランキングを出したいこともあるかもしれません。

title	price	publish	ランキング
SQL入門	5500	山田出版	1
PHPドリル	5100	山田出版	2
XML辞典	3300	翔泳社	3
XMLリファレンス	3200	日経BP	4
PHP5サンプル集	3000	秀和システム	5
PEAR入門	3000	翔泳社	5
SQLリファレンス	2500	日経BP	7
JSPリファレンス	1800	秀和システム	8
SQLプチブック	1600	日経BP	9
らくだの観察日記	1100	山田出版	10
フェレットの観察	1000	山田出版	11
ハムスターの観察	900	山田出版	12
かえるの観察日記	800	山田出版	13
あひるの観察日記	700	山田出版	14

そのような時のためにRANK関数以外にも順位付けの関数が用意されています

・RANK関数

ランキングを算出します。

同順位が複数レコード存在した場合、後続の順位が飛びます。

例) 1位が3レコードある場合 1位、1位、1位、4位・・・

・DENSE_RANK関数

おなじくランキングを算出します。

同順位が複数レコード存在しても、後続の順位は飛びません。

例) 1位が3レコードある場合 1位、1位、1位、2位・・・

・ROW_NUMBER関数

一意な連番を付与します。

例) 1位が3レコードある場合 1位、2位、3位、4位・・・

この他にもDBMSごとにデータ分析に役立つ様々なウィンドウ関数が用意されています(順位付け以外のものも)標準SQLに取り込まれている3つを紹介しました。

それぞれの実行結果はこのような感じです。

```

1  SELECT title,
2      price,
3      publish,
4      RANK() OVER (ORDER BY price DESC) AS "RANK",
5      DENSE_RANK() OVER (ORDER BY price DESC) AS "DENSE_RANK",
6      ROW_NUMBER() OVER (ORDER BY price DESC) AS "ROW_NUMBER"
7  FROM books;

```

title	price	publish	RANK	DENSE_RANK	ROW_NUMBER
SQL入門	5500	山田出版	1	1	1
PHPドリル	5100	山田出版	2	2	2
XML辞典	3300	翔泳社	3	3	3
XMLリファレンス	3200	日経BP	4	4	4
PHP5サンプル集	3000	秀和システム	5	5	5
PEAR入門	3000	翔泳社	5	5	6
SQLリファレンス	2500	日経BP	7	6	7
JSPリファレンス	1800	秀和システム	8	7	8
SQLプチブック	1600	日経BP	9	8	9
らくだの観察日記	1100	山田出版	10	9	10
フェレットの観察	1000	山田出版	11	10	11
ハムスターの観察	900	山田出版	12	11	12
かえるの観察日記	800	山田出版	13	12	13
あひるの観察日記	700	山田出版	14	13	14

ROW_NUMBERの列を見ると、価格が同じかどうかということは一切気にせずにひたすらに順番に連番を振っています。もし価格が同じレコードがあった場合にも、MySQLが適当な順序でレコードを並べます。ROW_NUMBER関数は順位づけではなく、レコードに一意的な連番を振りたいときに利用するとよいでしょう。

ウィンドウ関数には引数を何も与えていないことに気づいた人もいるかもしれません。RANK()やDENSE_RANK()、ROW_NUMBER()はカッコの中になにも書かず空のまま呼び出しされています。

実はウィンドウ専用関数を利用するときには、このようにカッコのなかには何も書かずに利用するという共通点があります。
後述する①集約関数をウィンドウ関数として使う との違いにもなります。

□ウィンドウ関数はどこで使うか

ここまで学習してきたSQLの関数の多くは、使える場所の制限はあまりありませんでした。

注意しなければいけないのが、WHERE句でSUM関数などの集約関数を使ってはいけないことくらいでした。

ウィンドウ関数も使える場所が大きく制限される関数の一つです。
というか1つの場所にしか書けません。

ウィンドウ関数を使用できるのは**SELECT**句です。
ウィンドウ関数はWHERE句やGROUP BY句の中では使用できません。

理由としては、ウィンドウ関数の実行順序が
WHERE や GROUP BY の処理が終わった後の結果に対して動作するように
作られているからです。

たとえば、先ほどのようなランキングを出すウィンドウ関数を使うおうと思ったときに
ランキングを出したのちに、WHEREでの絞り込みでレコードが減ってしまったり、GROUP BYによる集約が行われてしまったりは、せっかく出したランキングが水の泡、歯抜けのランキングが出来上がってしまうかもしれません。

このような都合から、ウィンドウ関数はそもそもSELECT句以外では「使う理由がない」ため、構文上も使えないように制限されているといった具合です。

□集約関数をウィンドウ関数として使う

次はSUMやAVGなどの集約関数をウィンドウ関数として使うパターンを学習しましょう。
構文はウィンドウ専用関数を使う時とまったく一緒です。

イメージをつかむためにSUM関数を使った例をやってみましょう。

```
1  SELECT title,
2      price,
3      publish,
4      SUM(price) OVER (ORDER BY price DESC) AS "累計"
5  FROM books;
```

title	price	publish	累計
SQL入門	5500	山田出版	5500
PHPドリル	5100	山田出版	10600
XML辞典	3300	翔泳社	13900
XMLリファレンス	3200	日経BP	17100
PHP5サンプル集	3000	秀和システム	23100
PEAR入門	3000	翔泳社	23100
SQLリファレンス	2500	日経BP	25600
JSPリファレンス	1800	秀和システム	27400
SQLプチブック	1600	日経BP	29000
らくだの観察日記	1100	山田出版	30100
フェレットの観察	1000	山田出版	31100
ハムスターの観察	900	山田出版	32000
かえるの観察日記	800	山田出版	32800
あひるの観察日記	700	山田出版	33500

← 5500
← 5500 + 5100
← 5500 + 5100 + 3300
← 5500 + 5100 + 3300 + 3200
:
:

SUM関数の場合、括弧の中に集計対象とする列を指定します。

今回はprice列を指定して合計を求めています。

ただし、レコード全体の合計値ではなく、1レコードずつ足し合わせた「累計」という
かたちで結果がでています。

Booksテーブルを題材に使用したので、今回は実務的にあまり使わないような結果に
なっていますが、売上データが記録されたテーブルなどで使用すれば
その時々売上金額の計算などで利用できそうです。

AVG関数を使用した例も見ましょう

```
1  SELECT title,
2      price,
3      publish,
4      AVG(price) OVER (ORDER BY price DESC) AS "現在の平均"
5  FROM books;
```


title	price	publish	現在の平均
SQL入門	5500	山田出版	5500.0000
PHPドリル	5100	山田出版	5300.0000
XML辞典	3300	翔泳社	4633.3333
XMLリファレンス	3200	日経BP	4275.0000
PHP5サンプル集	3000	秀和システム	3850.0000
PEAR入門	3000	翔泳社	3850.0000
SQLリファレンス	2500	日経BP	3657.1429
JSPリファレンス	1800	秀和システム	3425.0000
SQLプチブック	1600	日経BP	3222.2222
らくだの観察日記	1100	山田出版	3010.0000
フェレットの観察	1000	山田出版	2827.2727
ハムスターの観察	900	山田出版	2666.6667
かえるの観察日記	800	山田出版	2523.0769
あひるの観察日記	700	山田出版	2392.8571

$\leftarrow (5500) / 1$
 $\leftarrow (5500 + 5100) / 2$
 $\leftarrow (5500 + 5100 + 3300) / 3$
 $\leftarrow (5500 + 5100 + 3300 + 3200) / 4$
 $\vdots$
 $\vdots$

結果を見ると現在の平均列の計算は、たしかに平均ではありますがその集計対象となっているのは、「自分よりも上」のレコードだけであることがわかります。このように、「自分のレコード(カレントレコード)」を基準に集計対象を判断する点がウィンドウ関数として集約関数を使った場合の特徴となります。

PARTITION BYをつかって、ウィンドウを区切ってから使用することもできます。

□移動平均の算出

ウィンドウ関数は、テーブルをウィンドウという部分集合にカットして、その中で順位付けを行うものです。

しかし実はウィンドウの中でもさらに集計範囲を細かく設定するオプション機能があります。そのオプション機能で指定する集計範囲は「フレーム」と呼ばれています。

構文は、ORDER BYの後ろに範囲指定のキーワードを追加する形です。

```

1  SELECT title,
2      price,
3      publish,
4      AVG(price) OVER (ORDER BY price DESC
5                      ROWS 2 PRECEDING) AS "移動平均"
```

ROWS ロウズ と PRECEDING プレシーディング※前の を使って平均を算出する範囲を指定しています。

結果は次ページに記載

title	price	publish	移動平均
SQL入門	5500	山田出版	5500.0000
PHPドリル	5100	山田出版	5300.0000
XML辞典	3300	翔泳社	4633.3333
XMLリファレンス	3200	日経BP	3866.6667
PHP5サンプル集	3000	秀和システム	3166.6667
PEAR入門	3000	翔泳社	3066.6667
SQLリファレンス	2500	日経BP	2833.3333
JSPリファレンス	1800	秀和システム	2433.3333
SQLプチブック	1600	日経BP	1966.6667
らくだの観察日記	1100	山田出版	1500.0000
フェレットの観察	1000	山田出版	1233.3333
ハムスターの観察	900	山田出版	1000.0000
かえるの観察日記	800	山田出版	900.0000
あひるの観察日記	700	山田出版	800.0000

$\leftarrow (5500) / 1$
 $\leftarrow (5500 + 5100) / 2$
 $\leftarrow (5500 + 5100 + 3300) / 3$
 $\leftarrow (5500 + 5100 + 3300 + 3200) / 3$ ※※
 $\vdots$
 $\vdots$

この結果を最初のAVGのSQLと比較してみると、平均値列の内容が異なっているのがわかります。

赤く囲っている部分がフレームです。

フレームを指定したことで、平均の計算に使用するレコードの範囲が限定されています。

今回は **ROWS 2 PRECEDING** を使ったので

2行前までのレコードが集計範囲となっています
つまり、

- ・自分(カレントレコード)
- ・自分より1行前のレコード
- ・自分より2行前のレコード

という直近の3行分のレコードで平均値を計算している感じです。

4行目以降のレコードについても、同じように計算されており、その都度フレームが下にずれていくようになっています。

このような集計方法は一般に移動平均と呼ばれます。

その時々で「ここ最近の調子」を把握したい場合に便利ですので、株価のトレンドを時系列に追ったり、会計システムを開発する際に帳簿で使用したり、データ分析をしてマネジメントに役立てたりするときに利用できそうです。

またPRECEDINGの代わりに**FOLLOWING**(フォローイング)を利用すれば

- ・自分より1行前のレコード
- ・自分(カレントレコード)
- ・自分より1行後ろのレコード

といったフレームを指定することも可能となっています。BETWEEN演算子も使用します。

```

1  SELECT title,
2      price,
3      publish,
4      AVG(price) OVER (ORDER BY price DESC
5                      ROWS BETWEEN 1 PRECEDING AND 1 FOLLOWING) AS "移動平均"

```

□ORDER BY

ウィンドウ関数を使った時の並べ替えに関する注意点を紹介します。

今回紹介してきたSQLの中にはORDER BYが含まれていました。
表示された結果は一見priceを基準に並べられているように見えますが
実はこのウィンドウ関数の部分で指定しているORDER BYはウィンドウ関数が
どのような順序で計算するかを決めるだけの役割しかもっていません。

DBMSによっては、ウィンドウ関数で指定した順番でソートして結果を表示してくれる
ものもありますが(おそらくMySQLも)、すべてのDBMSでこれをしてくれるとは限りません。

どのDBMSでも動作する汎用的なSQLにしようと思えば
ORDER BYを2つ書くことで、明示的に並べ替えを指定しておくによさそうです。

```

1  SELECT title,
2      price,
3      publish,
4      RANK() OVER (PARTITION BY publish
5                  ORDER BY price DESC) AS "ランキング"
6  FROM books
7  ORDER BY "ランキング";

```

ORDER BYが1つのSQLの中で2回出てきて、違和感を感じるかもしれませんが、これで
結果の並び順が保障されます。